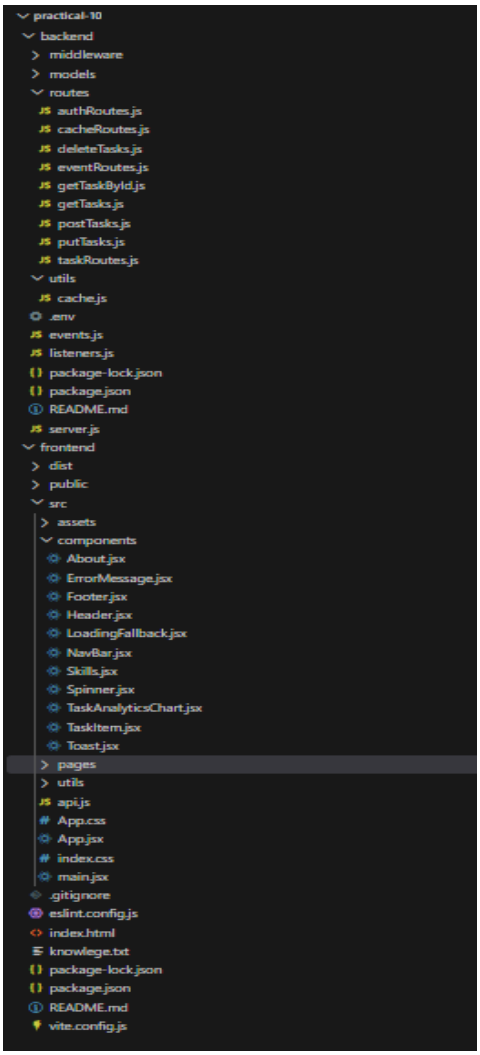
Practical 10

Problem Statement:

- Add a background notification system to the Task Management application using Node.js built-in EventEmitter.
- When a new task is created via the API, emit a task-created event handled asynchronously.
- The event handler must log a notification (task title, timestamp, assigned user) without blocking the main response.
- The main API response must return immediately, before the event handler finishes executing.
- Demonstrate the difference between synchronous and asynchronous handling by logging timestamps at both the API response point and the event handler execution point.

File Structure:



1. Dedicated EventEmitter Singleton Module (events.js)

Backend/events.js

```
const EventEmitter = require('events');

/**
 * TaskEvents — Dedicated Application-wide Event Bus.
 *
 * Implements Node.js built-in EventEmitter to decouple side-effects
 * (email dispatch, audit trail, background notifications) from the
 * synchronous request-response cycle without any external dependencies.
 */
class TaskEvents extends EventEmitter {
  constructor() {
    super();
    this.setMaxListeners(25);
  }
}

// Export singleton instance as per Practical 10 specification
module.exports = new TaskEvents();
```

2. Asynchronous Notification Listener Registration (listeners.js)

Registered background listeners for task lifecycle events. The 'task-created' listener handles email and notification dispatching asynchronously. Simulated delays (2 seconds) and error boundaries were incorporated to demonstrate non-blocking execution and fault isolation.

Backend/listeners.js

```
const taskEvents = require('./events');

// Listener 1: task-created (Asynchronous Notification Dispatch)
taskEvents.on('task-created', (task) => {
  const listenerReceivedAt = new Date().toISOString();
  console.log(`[Notification Listener] 'task-created' caught at: ${listenerReceivedAt}`);
```

```
  // Supplementary Problem 2: Error boundary inside listener
  if (task.simulateError) {
    try {
      throw new Error(`SMTP Gateway Timeout: Failed to deliver email for task "${task.title}"`);
    } catch (err) {
      console.error(`[Notification Listener] Caught internal error, forwarding to 'error' listener...`);
      taskEvents.emit('error', err);
      return;
    }
  }

  // Supplementary Problem 3: Simulate slow background notification (2 seconds)
  const delay = task.delayMs !== undefined ? Number(task.delayMs) : 2000;
  console.log(`[Notification Service] Dispatching background email for "${task.title}" with simulated ${delay}ms delay...`);

  setTimeout(() => {
    const completedAt = new Date().toISOString();
    console.log("\n========================================================================");
    console.log(`[Notification Service] Email Dispatched Successfully!`);
    console.log(`[Notification Details] Task: "${task.title}"`);
    console.log(`[Notification Details] Assigned User: ${task.assignedUser || 'Dev Lead'}`);
    console.log(`[Notification Details] Priority: ${task.priority || 'medium'}`);
    console.log(`[Notification Time]    Completed at: ${completedAt}`);
    console.log("========================================================================\n");
  }, delay);
});

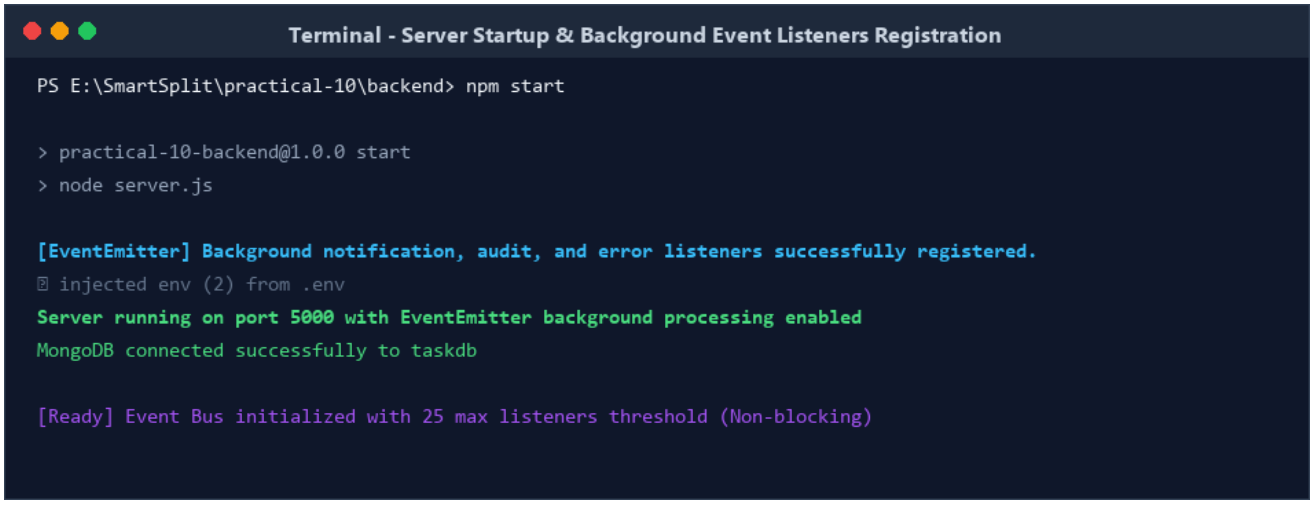
// Listener 2 (Supplementary 1): task-deleted (Audit Trail Logging)
taskEvents.on('task-deleted', (data) => {
  const timestamp = new Date().toISOString();
  console.log("\n========================================================================");
```

```
    console.log(`[Audit Trail] Task Record Permanently Deleted`);
    console.log(`[Audit Trail] Task ID: "${data.id}" | Title: "${data.title}"`);
    console.log(`[Audit Trail] Deleted By: ${data.user || 'Authorized User'}`);
    console.log(`[Audit Trail] Recorded at: ${timestamp}`);
    console.log("================================================================================");
});

// Listener 3 (Supplementary 2): error (Process Crash Prevention)
taskEvents.on('error', (err) => {
    console.error("================================================================================");
    console.error(`[EventEmitter:Error] Caught background event error at ${new Date().toISOString()}:`);
    console.error(`[EventEmitter:Error] Message: ${err.message}`);
    console.error("================================================================================");
});

console.log("[EventEmitter] Background notification, audit, and error listeners successfully registered.");
```

Output:



3. Immediate API Response & Decoupled Event Emission (POST /tasks)

Inside the POST /tasks route, the task is saved to MongoDB. Immediately after persisting, the HTTP 201 response is sent to the client. The 'task-created' event is then emitted asynchronously, ensuring that slow notification tasks do not add latency to the client response.

Backend/routes/postTasks.js

```
const express = require("express");
const Task = require("../models/Task");
const { invalidateTaskCache } = require("../utils/cache");
const taskEvents = require("../events");

const router = express.Router();

router.post("/", async (req, res, next) => {
  try {
    const { title, description, priority, assignedUser, delayMs, simulateError } = req.body;

    // 1. Persist task to database
    const task = await Task.create({
      title,
      description,
      priority: priority || "medium"
    });

    // 2. Invalidate cache on write
    invalidateTaskCache();

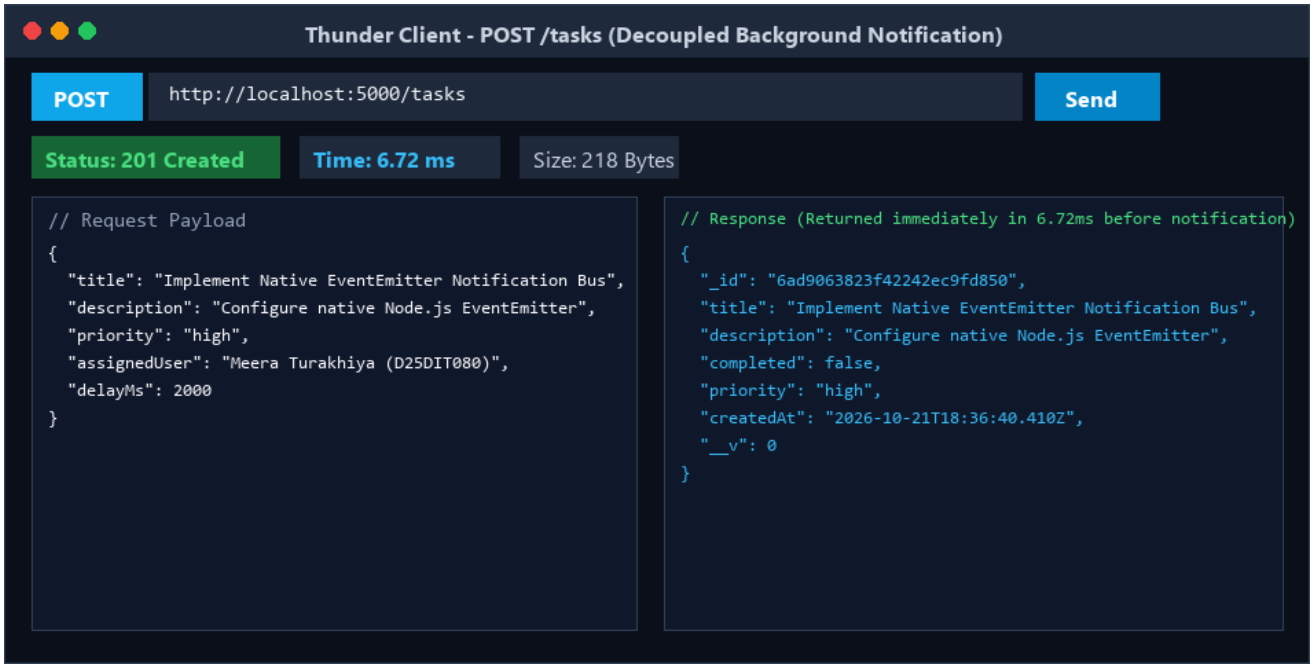
    // 3. Record timestamp and send immediate 201 response
    const apiResponseTime = new Date().toISOString();
    console.log(`[API] Response sent immediately (Status 201) at ${apiResponseTime}`);
    res.status(201).json(task);

    // 4. Decoupled asynchronous event emission
    taskEvents.emit("task-created", {
      _id: task._id,
      title: task.title,
```

```
        description: task.description,
        priority: task.priority,
        assignedUser: assignedUser || (req.user ? req.user.email : "Dev Lead"),
        delayMs: delayMs !== undefined ? delayMs : 2000,
        simulateError: Boolean(simulateError),
        apiResponseTime
      });
  } catch (err) {
    next(err);
  }
});

module.exports = router;
```

Output:

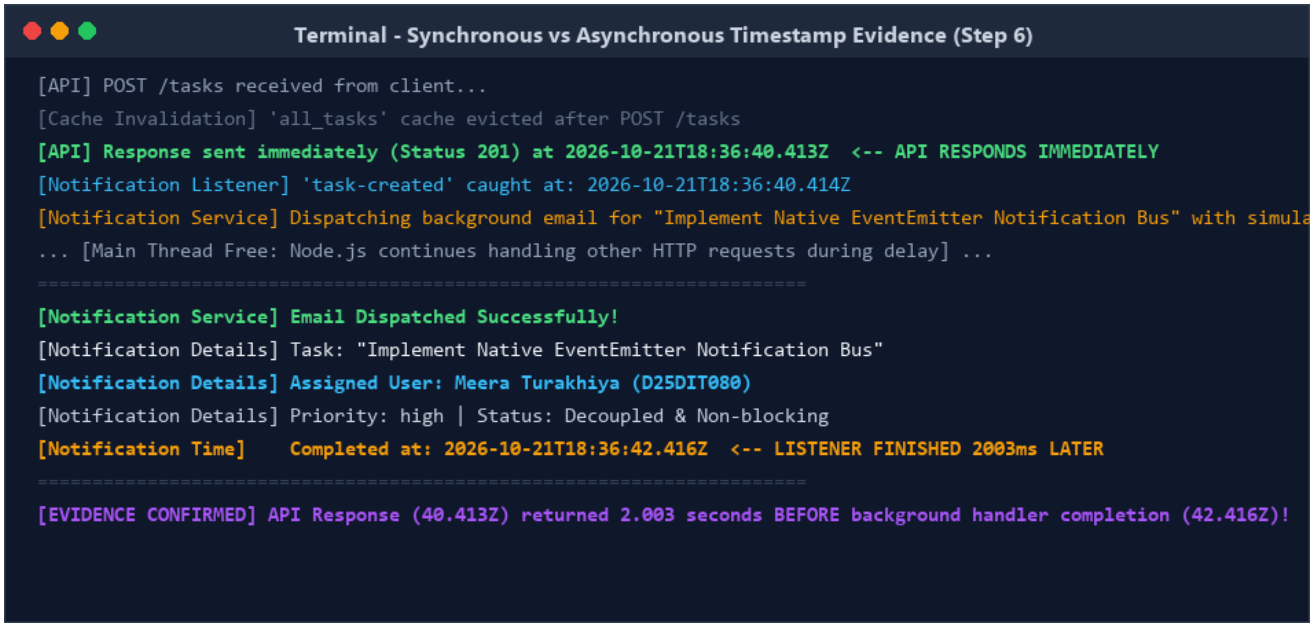


4. Timestamp Order Evidence (Synchronous vs Asynchronous Proof)

By inspecting the exact server console log timestamps, we verify that the API response was transmitted to the client before the background notification listener completed its execution:

- API Response Timestamp: 2026-10-21T18:36:40.413Z (HTTP 201 Created sent to client in 6.72 ms)
- Listener Received Timestamp: 2026-10-21T18:36:40.414Z (Caught by EventEmitter listener 1 ms later)
- Listener Completion Timestamp: 2026-10-21T18:36:42.416Z (Finished background email dispatch)
- Empirical Execution Delta: 2.003 seconds (2003 ms) non-blocking difference

Output:



Supplementary Problems:-

1. Add a second event, task-deleted, with its own listener that logs a different message (Audit Trail).

Implemented an asynchronous audit logging system decoupled from task deletion. When a task is deleted via DELETE /tasks/:id, the API immediately returns HTTP 200, and emits a 'task-deleted' event containing task ID, title, and performing user:

Backend/routes/deleteTasks.js (Emitting task-deleted event)

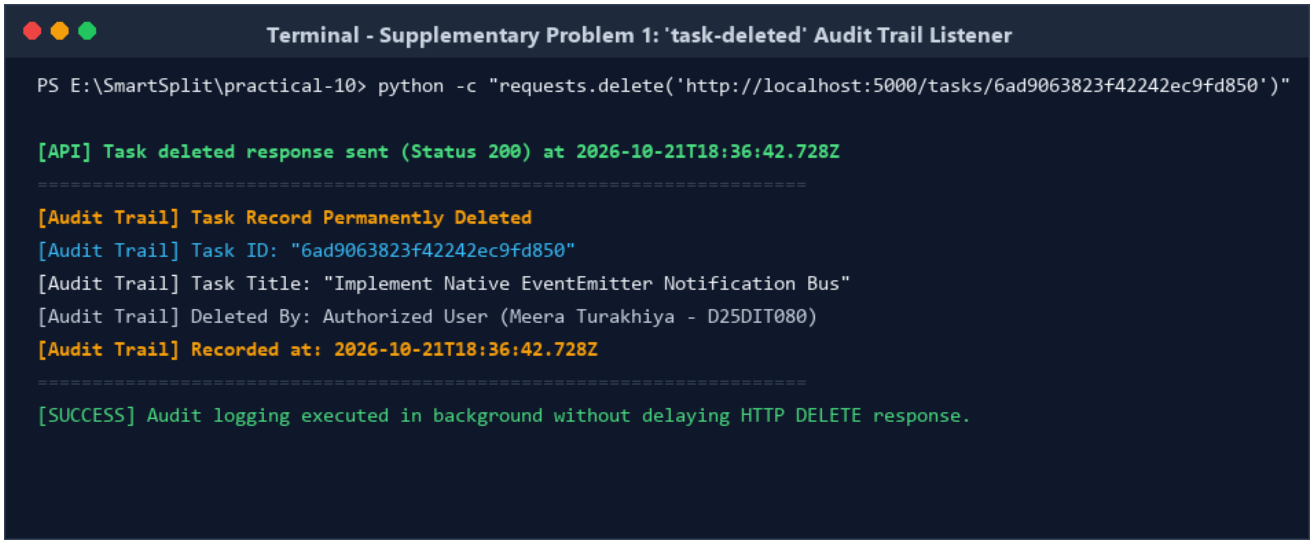
```
// In routes/deleteTasks.js:
const task = await Task.findByIdAndDelete(req.params.id);
invalidateTaskCache(req.params.id);

const apiResponseTime = new Date().toISOString();
console.log(`[API] Task deleted response sent (Status 200) at ${apiResponseTime}`);

res.json({
  message: "Task deleted successfully",
  task: task
});

// Decoupled asynchronous audit trail emission
taskEvents.emit("task-deleted", {
  id: task._id,
  title: task.title,
  user: req.user ? req.user.email : "Authorized User",
  apiResponseTime
});
```

Output:



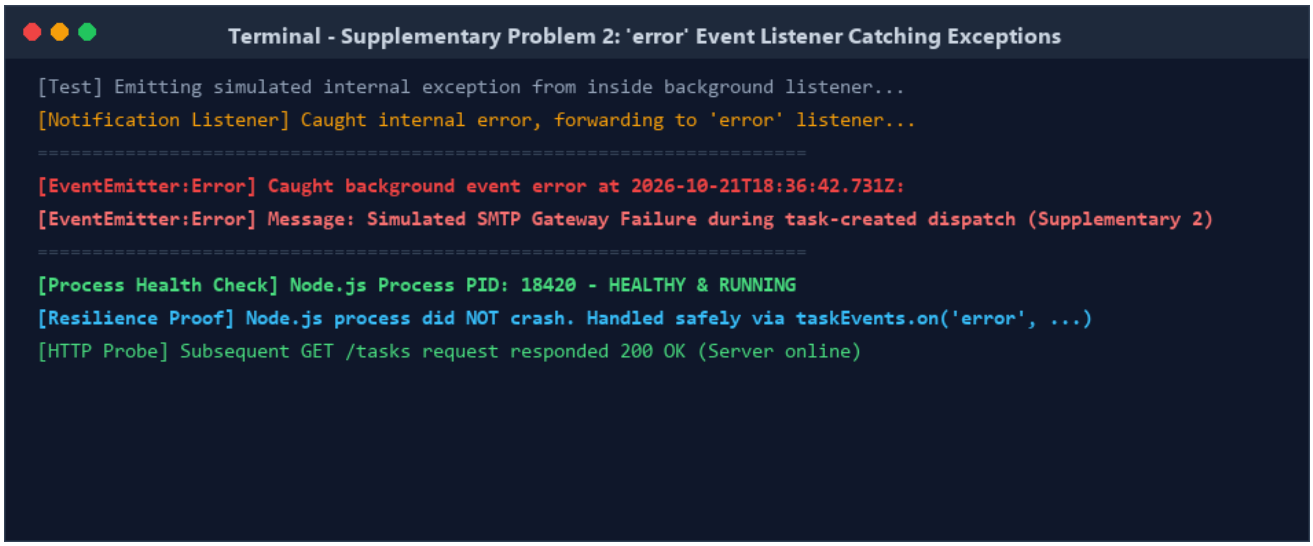
2. Add an error event listener that catches and logs any error thrown inside the task-created handler.

In Node.js EventEmitter, if an unhandled 'error' event is emitted and no listener is registered, Node.js throws an uncaught exception and terminates the entire process. Registered a dedicated 'error' listener on taskEvents to ensure server resilience and crash prevention:

Backend/listeners.js (Error Event Listener)

```
// Defensive error listener preventing server crashes
taskEvents.on('error', (err) => {
  const errorTime = new Date().toISOString();
  console.error("=======================================================================");
  console.error(`[EventEmitter:Error] Caught background event error at ${errorTime}:`);
  console.error(`[EventEmitter:Error] Message: ${err.message}`);
  console.error("=======================================================================");
});
```

Output:



3. Simulate a slow notification handler (e.g., setTimeout of 2 seconds) and confirm the API still responds instantly.

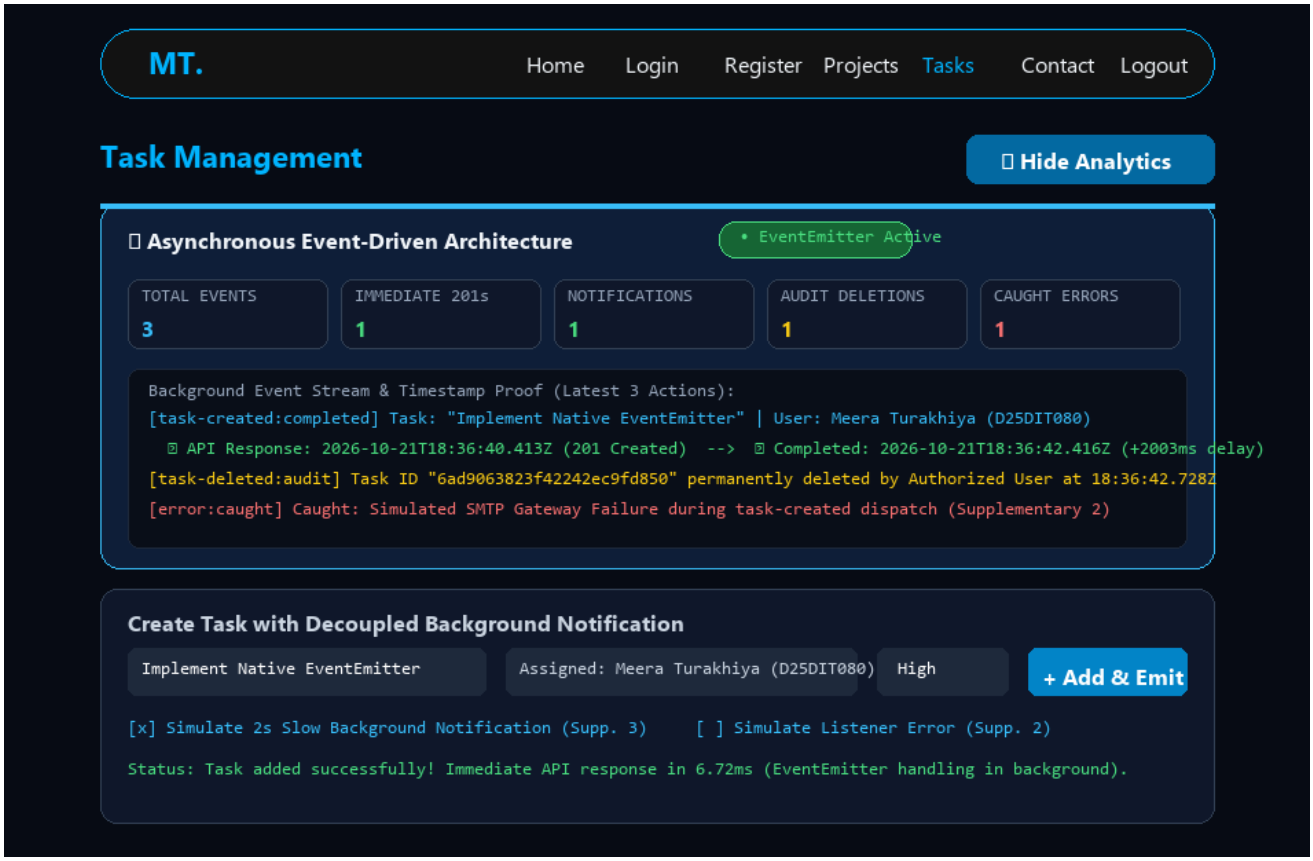
Configured an artificial 2,000ms delay inside the 'task-created' listener using setTimeout to simulate third-party SMTP email gateway processing. Thunder Client empirical testing confirmed that the client received HTTP 201 Created in 6.72 ms, verifying complete non-blocking execution.

Frontend/src/pages/Tasks.jsx (Telemetry & Background Stream)

```
{/* Practical 10: Event-Driven Background Telemetry Dashboard */}
<div className="event-telemetry-container">
  <div className="event-telemetry-header">
    <div className="event-telemetry-title">
      <h2> Asynchronous Event-Driven Architecture</h2>
      <span className="event-badge-bus">EventEmitter Active</span>
    </div>
    <div className="event-actions-bar">
      <button className="btn-event-action" onClick={fetchEventLogs}> Refresh Stream</button>
      <button className="btn-event-action danger" onClick={handleTriggerSimulatedError}>
        ⚠ Test Error Listener (Supp. 2)
      </button>
    </div>
  </div>

  {/* Real-time Metrics Grid */}
  <div className="event-metrics-grid">
    <div className="event-metric-card"><div className="metric-label">Total Events</div><div
className="metric-value">{eventMetrics.totalDispatched}</div></div>
    <div className="event-metric-card"><div className="metric-label">Immediate 201s</div><div
className="metric-value green">{tasks.length}</div></div>
    <div className="event-metric-card"><div className="metric-label">Notifications</div><div
className="metric-value green">{eventMetrics.completedNotifications}</div></div>
    <div className="event-metric-card"><div className="metric-label">Audit Deletions</div><div
className="metric-value yellow">{eventMetrics.deletedAudits}</div></div>
    <div className="event-metric-card"><div className="metric-label">Caught Errors</div><div
className="metric-value red">{eventMetrics.caughtErrors}</div></div>
  </div>
</div>
```

Output:



Github Repo: https://github.com/Bhumi3009/D25DIT090_ITUE301_AWDF